

大模型量化技术：从 INT8 到 INT4 的工程实践

目 录

1 引言：为什么需要量化	3
1.1 显存墙：量化的第一驱动力	3
1.2 算力与带宽：量化的第二驱动力	3
1.3 三种量化路径预览	3
2 量化数学基础	4
2.1 线性量化的基本公式	4
2.2 对称量化与非对称量化	4
2.2.1 对称量化	4
2.2.2 非对称量化	4
2.3 量化粒度	5
2.4 量化误差的来源	5
2.5 误差衡量指标	6
3 INT8 权重-激活量化 (W8A8)	6
3.1 为什么同时量化权重和激活	6
3.2 per-token 与 per-matrix 的不对称粒度	7
3.3 INT8 点积的累加位宽	7
3.4 requantize: SwiGLU 中的精度转换	7
4 INT8 逐级量化模拟 FP64	8
4.1 与 W8A8 的本质区别	8
4.2 逐级量化分解的数学	8
4.3 量化层数截断优化	9
5 INT4 权重量化 (W4A16) 与 GPTQ	9
5.1 W4A16 与 W8A8 的对比	9
5.2 INT4 打包格式	10
5.3 RTN：最简单的基线及其不足	10
5.4 GPTQ：二阶误差补偿	11
5.4.1 Hessian 矩阵的物理意义	11
5.4.2 Cholesky 分解与误差传播	11
5.4.3 dead columns 与校准数据	12
5.5 W4A16 的显存分析	12
6 量化推理的瓶颈分析	12
6.1 反量化瓶颈	12

6.2 融合 dequant-GEMM 原理	13
6.3 HybridQuantizedLinear: 按 M 切换策略	13
6.4 Lab 2 的 requantize 瓶颈对比	13
7 量化格式全景与前沿	14
7.1 四类量化格式	14
7.2 NVFP4: 两级缩放的 4 bit 浮点	14
7.3 异常值问题	14
8 跨实验对比与总结	15
8.1 统一对比表	15
8.2 量化方法选择逻辑	16
9 本章你将学会	16
10 要点速查	17
11 小结	18

1 引言：为什么需要量化

量化（quantization）是 HPC 实验中反复出现的核心技巧。在 Lab 2 中，我们用 INT8 量化把 MoE 推理的矩阵乘法加速了 9.5 倍；在 Lab 4.5 中，我们用 INT8 逐级量化模拟 FP64 矩阵乘法，实现了 36.8 倍加速；在 Lab 5 中，我们用 GPTQ INT4 量化把一个 120 亿参数的模型从 24 GiB 压缩到 7.2 GiB，塞进了 10 GiB 的显存限制。三个实验，三种量化路径，但它们共享同一套数学基础和设计哲学。

本章的目标是把这些分散在各个实验中的量化知识整合起来，从原理层面讲清楚：量化在做什么，为什么有效，不同方法各自的取舍是什么。

1.1 显存墙：量化的第一驱动力

以 Lab 5 的 Gemma4-12B 为例。这个模型有约 120 亿参数，BF16 下每个参数占 2 字节，仅权重就需要约 24 GiB 显存。但实验分配的 GPU 只有 10 GiB（H800 MIG 10G），连模型权重都装不下，更不要说推理时的 KV cache 和激活了。

直觉 量化的本质是降低数值精度来换取存储和带宽。把 BF16（16 bit）降到 INT4（4 bit），每个参数从 2 字节缩到 0.5 字节，权重显存从 24 GiB 降到约 6 GiB。这就像把一本精装字典换成口袋本，内容大致不变但体积缩小了四分之三。代价是精度损失：INT4 只有 16 个离散值，无法精确表示 BF16 的连续浮点数。如何控制这个精度损失，就是量化技术的核心问题。

1.2 算力与带宽：量化的第二驱动力

量化不只省显存，还能提速。原因有两层：

1. **降低 GEMM 的计算量**：INT8 GEMM 的吞吐通常是 FP16 的 2 倍，INT4 更高。现代 GPU 和 CPU 都有专用的低精度指令（如 Intel AMX、ARM SVE、NVIDIA Tensor Core），在低精度下能输出更多 FLOPs。
2. **降低内存带宽**：推理的 Decode 阶段是访存密集的，每步都要读取全部权重。权重从 16 bit 降到 4 bit，读取量减少 4 倍，带宽瓶颈直接缓解。

在 Lab 5 中，实测反量化（dequant）的总 CUDA 时间约 2.2 s，而实际 GEMM 仅 0.22 s，两者比例约 10:1。这说明如果反量化不优化，量化省下的带宽又被反量化吃回去了。这正是融合 dequant-GEMM 内核的动机所在。我们将在第六章详细分析。

1.3 三种量化路径预览

实验	量化方法	位宽	核心思路
Lab 2	W8A8（权重加激活）	INT8	同时量化权重和激活，用 AMX 原生 INT8 GEMM 加速 MoE 推理
Lab 4.5	逐级量化分解	INT8	把 FP64 分解为多个 INT8 分量，用 INT8 Tensor Core 模拟 FP64 计算
Lab 5	W4A16（仅权重）加 GPTQ	INT4	只量化权重不动激活，用二阶补偿控制 INT4 精度损失

这三条路径代表了量化的三种典型用法：压缩模型、加速计算、模拟高精度。它们的数学基础是共通的，但在粒度选择、误差补偿和系统优化上各有侧重。

2 量化数学基础

无论 INT8 还是 INT4，无论量化权重还是激活，所有线性量化方法都建立在同一套数学框架之上。我们从最基本的公式讲起。

2.1 线性量化的基本公式

给定一个实数 r (real 值，即原始的浮点数)，线性量化把它映射到一个低精度整数 q ：

$$q = \text{round}\left(\frac{r}{s} + z\right) \quad (2.1)$$

反量化时，从整数 q 恢复近似实数：

$$r \approx \hat{r} = s \cdot (q - z) \quad (2.2)$$

其中 s 是**缩放因子** (scale)，决定量化网格的间距； z 是**零点** (zero point)，决定实数 0 映射到哪个整数。 q 的取值范围由位宽决定：INT8 对称量化时 $q \in [-128, 127]$ ，INT4 对称量化时 $q \in [-8, 7]$ 。

直觉 想象一把尺子： s 是刻度间距， z 是零刻度的位置。把一个实数对准最近的刻度读数就是量化 (round 操作)，把刻度读数乘以间距就是反量化。刻度越粗 (s 越大)，能覆盖的范围越宽但误差越大；刻度越细 (s 越小)，精度越高但能表示的范围越窄。量化的核心矛盾就是： s 必须足够大以覆盖数据范围，又必须足够小以控制误差。

2.2 对称量化与非对称量化

2.2.1 对称量化

对称量化 (symmetric quantization) 令零点 $z = 0$ ，量化值关于 0 对称。INT4 对称量化的整数范围为 $[-8, 7]$ ，共 16 个值。缩放因子取数据组中绝对值最大的元素除以整数范围上限：

$$s = \frac{\max(|w|)}{7} \quad (\text{INT4}), \quad s = \frac{\max(|w|)}{127} \quad (\text{INT8}) \quad (2.3)$$

反量化时 $\hat{r} = s \cdot q$ ，不需要减零点，计算简单。

对称量化的好处是 $z = 0$ ，反量化只需一次乘法，没有额外存储开销。坏处是如果数据分布不对称 (比如 ReLU 后的激活值全为正数)，会浪费一半的量化范围。

2.2.2 非对称量化

非对称量化 (asymmetric quantization) 允许 $z \neq 0$ ，量化值范围为 $[0, 15]$ (INT4) 或 $[0, 255]$ (INT8)。缩放因子和零点的计算为：

$$s = \frac{\max(w) - \min(w)}{15}, \quad z = \text{round}\left(-\frac{\min(w)}{s}\right) \quad (2.4)$$

反量化时 $\hat{r} = s \cdot (q - z)$ ，需要先减零点再乘缩放因子。

非对称量化充分利用了全部量化范围，对于不对称的数据分布精度更好。但需要额外存储 z ，反量化时多一次减法。

例 取权重 $w = [0.3, -1.2, 0.5, 2.1]$ 做 INT4 对称量化, $\text{group_size} = 4$ (整组共享一个 scale)。

对称量化: $s = \frac{\max(|w|)}{7} = \frac{2.1}{7} = 0.3$, $q = \text{round}(\frac{w}{0.3}) = \text{round}([1.0, -4.0, 1.67, 7.0]) = [1, -4, 2, 7]$ 。反量化后 $\hat{w} = 0.3 \times [1, -4, 2, 7] = [0.3, -1.2, 0.6, 2.1]$, 最大误差 0.1 (在 $w_0 = 0.5$ 处)。

如果改用非对称量化: $s = \frac{2.1 - (-1.2)}{15} = 0.22$, $z = \text{round}(\frac{1.2}{0.22}) = 5$ 。 $q = \text{round}(\frac{w}{0.22} + 5) = \text{round}([6.36, -0.45, 7.27, 14.55]) = [6, 0, 7, 15]$ 。反量化后 $\hat{w} = 0.22 \times ([6, 0, 7, 15] - 5) = [0.22, -1.1, 0.44, 2.2]$, 最大误差 0.1 (在 $w_2 = 0.5$ 处)。

这个例子中两者误差接近, 因为数据范围大致对称。但如果数据全是正数 (如 $w = [0.5, 1.0, 1.5, 2.0]$), 对称量化浪费了 $[-8, -1]$ 共 8 个值, 而非对称量化能用满 $[0, 15]$ 。

2.3 量化粒度

一个权重矩阵有上百万个参数, 它们的数据范围可能差异很大。如果整个矩阵共用一组 s 和 z , 有些通道的值范围小, 却被迫用很大的 s , 精度损失严重。把粒度变细, 让每组通道有自己的 s 和 z , 可以更贴合数据分布。

粒度	共享范围	特点	实验使用
per-tensor	整个张量	开销最小, 精度损失最大	Lab4.5 (FP64 模拟)
per-matrix	每个权重矩阵	权重静态分布, 一个 scale 足够	Lab2 (权重)
per-token	每个输入 token	激活动态变化, 逐 token 适配	Lab2 (激活)
per-group	每 G 列	精度与开销的折中	Lab5 ($G = 128$)
per-channel	每行或每列	精度好但 scales 数量多	理论参考

直觉 粒度越细, scale 越贴合数据, 精度越高, 但需要存储更多 scale 参数。per-tensor 只存一个 scale, 但一刀切; per-group 存 $\frac{d}{G}$ 个 scale, 折中。Lab 5 取 $G = 128$ 是 INT4 量化的经验值: 再细, scale 参数的存储开销开始抵消量化省下的空间; 再粗, INT4 的精度损失不可接受。

Lab 2 的设计很精妙: 权重用 *per-matrix scale* (因为权重在推理期间不变, 分布固定, 一个矩阵级 scale 足够), 激活用 *per-token scale* (因为不同 token 的激活幅值可能相差很大, 逐 token 计算能利用完整的 INT8 动态范围)。这是一个不对称的粒度策略, 针对权重和激活的不同特性做了专门优化。

2.4 量化误差的来源

量化误差有三个来源:

1. **舍入误差**: $q = \text{round}(\frac{r}{s} + z)$ 的 round 操作引入的 $[-0.5, 0.5]$ 整数级误差。反量化后变为 $[-\frac{s}{2}, \frac{s}{2}]$ 的浮点误差。 s 越大, 舍入误差越大。
2. **截断误差**: 如果 $\frac{r}{s} + z$ 超出整数范围 (如 INT4 的 $[-8, 7]$), 需要截断到边界, 引入更大的误差。这发生在数据中有异常值时。
3. **分布失配**: 如果 scale 的计算 (如 $s = \frac{\max(|w|)}{7}$) 被少数极端值主导, 大部分正常值被迫用很大的 s , 精度严重下降。这是大模型量化的核心难题。

2.5 误差衡量指标

不同实验用不同的指标衡量量化误差:

指标	定义	使用实验
NLL	$\text{NLL} = -\frac{1}{L-1} \sum \log p(x_{t+1} x_1, \dots, x_t)$, 衡量模型预测能力	Lab5
ΔNLL	$\text{NLL}_{\text{INT4}} - \text{NLL}_{\text{BF16}}$, 量化前后预测能力差异	Lab5
MSE	$\frac{1}{N} \sum (r - \hat{r})^2$, 逐元素均方误差	Lab2, Lab4.5
L2 relative error	$\frac{\ r - \hat{r}\ }{\ r\ }$, 相对误差	Lab2, Lab4.5
RMSE	$\sqrt{\text{MSE}}$, 均方根误差	Lab2

NLL 是语言模型的标准指标, 直接衡量量化对模型理解能力的影响。Lab 5 要求 $\Delta \text{NLL} < 0.16$, 即量化后模型的预测概率分布不能偏离太多。MSE 和 L2 relative error 则是数值层面的指标, 衡量量化前后矩阵元素的逐点误差。Lab 2 的精度目标是 *per-token relative L2* $< 2e-2$, *global RMSE* $< 2e-3$; Lab 4.5 的 L2 相对误差达到 $1.22e-7$ 。

3 INT8 权重-激活量化 (W8A8)

Lab 2 的场景是 MoE 推理优化。MoE 模型的核心计算是专家前馈网络 (Expert FFN), 它包含 gate、up、down 三个线性层加 SwiGLU 激活。当 batch size 和专家数增大时, FP32 或 BF16 的矩阵乘法成为瓶颈。Lab 2 的解法是 W8A8: 把权重和激活都量化到 INT8, 用 Intel AMX 指令的原生 INT8 GEMM 加速。

3.1 为什么同时量化权重和激活

W8A8 和 Lab 5 的 W4A16 有一个根本区别: W8A8 量化权重和激活, GEMM 直接在 INT8 域进行; W4A16 只量化权重, 激活保持 FP16, GEMM 时需要先把 INT4 权重反量化到 FP16 再计算。

直觉 为什么 Lab 2 选择 W8A8 而不是 W4A16? 因为目标不同。Lab 2 的瓶颈是算力 (MoE FFN 的 GEMM 太多), 需要用 INT8 GEMM 直接加速计算。W8A8 让两个 INT8 矩阵直接相乘, 利用 AMX 的 INT8 吞吐优势。而 Lab 5 的瓶颈是显存 (模型装不下), W4A16 只需压缩权重存储, GEMM 仍用 FP16 累加, 不依赖 INT4 GEMM 硬件支持。选择哪种方案, 取决于瓶颈在算力还是在显存。

3.2 per-token 与 per-matrix 的不对称粒度

Lab 2 对权重和激活采用了不同的量化粒度，这是一个精妙的设计：

1. **权重用 per-matrix scale**：权重在推理期间保持不变，同一个矩阵的数值分布固定，因此在此在预处理阶段只计算并保存一个矩阵级 scale（如 s_{gate} 、 s_{up} 、 s_{down} ）。更细的 per-row 或 per-channel 权重量化可能进一步降低误差，但本实验的数据格式只提供每矩阵一个 scale。
2. **激活用 per-token scale**：不同 token 的激活幅值可能相差很大，逐 token 计算能利用完整的 INT8 动态范围。per-token scale 使用当前 token 的最大绝对值：

$$s_{x,t} = \max_i \frac{|x_{t,i}|}{127} \quad (3.1)$$

这样每个 token 都能使用接近完整的 INT8 动态范围 $[-128, 127]$ 。

这个不对称设计反映了一个深刻的不对称性：权重是静态的，一次量化终身使用；激活是动态的，每次推理都不同。静态的可以用粗粒度（per-matrix），因为分布已知且固定；动态的必须用细粒度（per-token），否则某些 token 的激活幅值小，被全局 scale 淹没。

3.3 INT8 点积的累加位宽

INT8 量化的一个关键工程问题是：两个 INT8 数相乘后，乘积必须在哪里累加？

单个 $\text{INT8} \times \text{INT8}$ 乘积的最大绝对值是多少？完整 INT8 范围为 $[-128, 127]$ ，最保守的单项乘积绝对值上界为：

$$|(-128) \times (-128)| = 16384 = 2^{14} \quad (3.2)$$

如果用 INT16 累加（正数上限 $32767 = 2^{15} - 1$ ），最坏情况下累加到第 2 项就会溢出（ $2 \times 16384 = 32768 > 32767$ ）。因此 INT8 点积**必须**在 INT32 中累加。

例 INT32 累加器的保守上界分析：假设向量维度 $d = 4096$ （大模型隐藏维度的典型值），最坏情况下累加 d 项：

$$\max \text{accumul} = 4096 \times 2^{14} = 2^{12} \times 2^{14} = 2^{26} \quad (3.3)$$

INT32 正上限为 $2^{31} - 1 \approx 2.1 \times 10^9$ ， $2^{26} \approx 6.7 \times 10^7$ ，占 INT32 正上限约 $\frac{1}{32}$ ，余量充足。这就是为什么 AMX 和 VNNI 指令都设计为 INT8 输入、INT32 累加：既保证不溢出，又不浪费累加器位宽。

3.4 requantize: SwiGLU 中的精度转换

MoE 的专家 FFN 使用 SwiGLU 激活： $h = \text{SwiGLU}(xW_{\text{gate}}) \cdot (xW_{\text{up}})$ ，然后 $y = hW_{\text{down}}$ 。计算流程是：输入 x (INT8) $\rightarrow$ gate 投影 (INT8 GEMM) $\rightarrow$ 中间结果 (需要激活函数) $\rightarrow$ up 投影 (INT8 GEMM) $\rightarrow$ 乘积 $\rightarrow$ down 投影 (INT8 GEMM) $\rightarrow$ 输出。

问题在于：INT8 GEMM 输出的是 INT32 累加结果，但 SwiGLU 激活函数（Silu 和逐元素乘法）是浮点运算，不能在 INT8 域完成。因此需要在每个阶段之间做**重量化**（requantize）：把 INT32 结果反量化为浮点，执行激活，再量化回 INT8。

直觉 requantize 就像翻译：两个说不同语言的人通过翻译交流。INT8 GEMM 说整数语言，SwiGLU 说浮点语言，中间需要一个翻译（requantize）来转换。每次翻译都会损失一点信息（量化误差），但这是不可避免的，因为激活函数的非线性运算无法在整数域精确表达。Lab 2 的性能分析显示 requantize 是热点之一，在优化后仍占 expert FFN 时间的相当比例。

Lab 2 实测 S3（16 专家）配置下达到 9.5 倍加速，其中 expert FFN 占 74.59%。这意味着量化后的 GEMM 已经很快了，但 requantize 和 SwiGLU 仍然是下一步优化的目标。这也解释了为什么 W4A16（Lab 5）选择了不同路线：它不做 requantize，激活全程在 FP16 域，权重在 GEMM 时反量化。

4 INT8 逐级量化模拟 FP64

Lab 4.5 的场景与 Lab 2 和 Lab 5 完全不同。这里的目标不是压缩模型或加速推理，而是用 INT8 Tensor Core 来模拟 FP64 矩阵乘法。FP64 GEMM 在 GPU 上很慢（Tensor Core 原生支持 FP64），但 INT8 Tensor Core 的吞吐远高于 FP64 CUDA Core。如果能用多个 INT8 分量逼近 FP64 的精度，就能获得巨大的加速。

4.1 与 W8A8 的本质区别

W8A8 量化（Lab 2）是**压缩**：把高精度数据降到低精度，接受精度损失以换取速度。逐级量化（Lab 4.5）是**模拟**：把高精度数据分解为多个低精度分量的组合，通过精心设计的叠加来恢复高精度。前者丢信息，后者保留信息。

直觉 想象用黑白打印机印彩色照片。W8A8 像直接转灰度：信息丢了但够用。逐级量化像分色印刷：用青、品红、黄、黑四色叠加，每一色都是低精度（有或无），但叠加起来逼近全彩色。每一级 INT8 分量捕捉原始数据的一个精度层级，叠加越多层精度越高。

4.2 逐级量化分解的数学

FP64 具有 53 bit 有效尾数，而一个 INT8 分量只能表达 $[-127, 127]$ 的整数范围。逐级量化把矩阵元素 x 近似写成多个 INT8 分量的加权和：

$$x \approx \sum_{i=0}^{S-1} q_i s_i, \quad q_i \in [-127, 127] \cap \mathbb{Z} \quad (4.1)$$

其中 q_i 是第 i 级 INT8 分量， s_i 是全矩阵共享的 FP64 比例尺， S 是分解层数（split 数）。

关键在于 scale 的递推关系。令 $r_0 = x$ 为原始值，初始比例尺由矩阵最大绝对值 $X_{\max}$ 决定：

$$s_0 = \frac{X_{\max}}{127}, \quad q_0 = \text{round}\left(\frac{r_0}{s_0}\right) \quad (4.2)$$

残差为 $r_1 = r_0 - q_0 s_0$ ，下一级比例尺缩小 254 倍：

$$s_{i+1} = \frac{s_i}{254}, \quad q_{i+1} = \text{round}\left(\frac{r_{i+1}}{s_{i+1}}\right), \quad r_{i+2} = r_{i+1} - q_{i+1} s_{i+1} \quad (4.3)$$

直觉 为什么是 254？因为 INT8 对称范围是 $[-127, 127]$ ，最大绝对值 127。前一级 scale s_i 捕捉了 $[-127s_i, 127s_i]$ 的范围，残差 r_{i+1} 的最大值不超过 $\frac{s_i}{2}$ （舍入误差上界）。为了让下一级 q_{i+1} 能利用 $[-127, 127]$ 的完整范围， s_{i+1} 应取 $\frac{s_i}{2 \times 127} = \frac{s_i}{254}$ 。这样每一级都从前一级残差中提取新的有效信息，比例尺按 254 缩小，精度逐级提升。

例 以 $S = 4$ 为例，假设 $X_{\max} = 1.0$ ：

$$s_0 = \frac{1.0}{127} \approx 7.87 \times 10^{-3} \quad (4.4)$$

$$s_1 = \frac{s_0}{254} \approx 3.10 \times 10^{-5} \quad (4.5)$$

$$s_2 = \frac{s_1}{254} \approx 1.22 \times 10^{-7} \quad (4.6)$$

$$s_3 = \frac{s_2}{254} \approx 4.80 \times 10^{-10} \quad (4.7)$$

FP64 的机器精度约为 1.1×10^{-16} ，4 级分解后最细 scale 达到 10^{-10} 量级。虽然还达不到 FP64 全精度，但配合剪枝（prune_d）和 INT8 Tensor Core 的高吞吐，已经能在实测中超越原生 FP64 cuBLAS。

4.3 量化层数截断优化

Lab 4.5 的一个关键优化是 quant_splits。在 prune_d=3 的配置下，只有 split 层 0、1、2 被任何保留的 GEMM pair 使用。但原始实现仍对所有 splits 层执行量化，将从未被读取的 INT8 分量写入显存。对 splits=8，8 层中有 5 层（62.5%）完全浪费。

优化方案是 quant_splits = min(splits, prune_d)，只量化实际使用的层。预期量化时间与 splits 参数解耦，splits=8 时量化时间下降约 60%。

Lab 4.5 实测量化时间下降 61%，量化写入量从 splits 份 INT8 降至 3 份。Nsight Systems 分析显示瓶颈从 DRAM（86.64%，写太多）翻转为 SM（73.52%，计算为主）。这是一个典型的“减少无用功”优化：不做任何计算上的改进，只是跳过不必要的工作。最终 8192³ 规模下达到 36.81 倍加速，L2 相对误差仅 1.22e-7。

5 INT4 权重量化（W4A16）与 GPTQ

Lab 5 的场景是 LLM 推理的显存压缩。Gemma4-12B 有约 120 亿参数，BF16 下需要 24 GiB，但 GPU 只有 10 GiB。INT4 量化把每个权重压缩到 4 bit（0.5 字节），权重显存降到约 6 GiB，加上其他开销总共约 7.2 GiB，勉强塞进 10 GiB。

5.1 W4A16 与 W8A8 的对比

维度	W8A8 (Lab2)	W4A16 (Lab5)
量化对象	权重 + 激活	仅权重

GEMM 精度	$\text{INT8} \times \text{INT8} \rightarrow \text{INT32}$	$\text{INT4} \rightarrow \text{FP16}$ 反量化 $\times$ FP16 激活
GEMM 硬件	AMX / VNNI (原生 INT8)	FP16 Tensor Core (通用)
requantize	需要 (SwiGLU 之间)	不需要 (激活全程 FP16)
主要瓶颈	算力	显存
精度指标	$\text{RMSE} < 2\text{e-}3$	$\Delta \text{NLL} < 0.16$

直觉 W4A16 的核心权衡是：只量化权重，不量化激活。好处是激活保持 FP16 精度，GEMM 用 FP16 Tensor Core (广泛支持且精度好)，不需要 requantize。坏处是 GEMM 前必须先把 INT4 权重反量化为 FP16 (dequant)，这引入了额外开销。Lab 5 实测反量化开销是 GEMM 的 10 倍，成为首要瓶颈。这是一个“省了显存但多了计算”的代价。

5.2 INT4 打包格式

INT4 量化后每个权重占 4 bit，但内存按字节 (8 bit) 寻址。Lab 5 采用 `uint8_little_nibble` 格式：把两个 INT4 值打包进一个 `uint8`，低 nibble (低 4 bit) 存放偶数索引的权重，高 nibble (高 4 bit) 存放奇数索引的权重。

编码时先把 $q \in [-8, 7]$ 加 8 映射到 $[0, 15]$ ，然后两个值按位或合并：`packed = low | (high << 4)`。反量化时用位运算提取：`low = packed & 0x0F`，`high = (packed >> 4) & 0x0F`，再减 8 还原到 $[-8, 7]$ 。

打包格式的选择影响反量化效率。Lab 5 的优化版本用无分支位运算替代条件分支：`(packed >> ((k_offs & 1) * 4)) & 0xF`，用移位量自动选择低或高 nibble，避免了 `tl.where` 的条件分支开销。这是一个小但重要的性能优化，在融合 `dequant-GEMM kernel` 中显著提升了吞吐。

5.3 RTN：最简单的基线及其不足

RTN (Round to Nearest, 最近舍入) 是最简单的量化方法：对每个权重组独立计算 s 和 z ，然后直接舍入。它不考虑输入数据的分布，只看权重本身。

RTN 的流程是：把权重矩阵按列分成若干组，每组 G 列；对每组计算 $s = \frac{\max(|w|)}{7}$ (对称量化)；对每个权重 $q = \text{round}(\frac{w}{s})$ ，截断到 $[-8, 7]$ 。

直觉 RTN 的根本缺陷是：它把每个权重视为独立的，量化第 j 列的误差不会影响第 $j+1$ 列的决策。实际上，权重矩阵的各列通过输入激活产生关联，一列的量化误差会影响其他列的输出，进而影响整体精度。这就像排队时每个人独立决定站哪里，不考虑前一个人的位置，整体排列会很乱。

Lab 5 的实测数据清楚地展示了 RTN 的不足：

方法	mean_nll	ΔNLL	通过阈值 0.16?
BF16 (参考)	2.3085	-	-
RTN	2.6128	0.3044	否

GPTQ (修正后)	2.4015	0.0930	是
------------	--------	--------	---

RTN 的 ΔNLL 为 0.3044，远超阈值 0.16，而 GPTQ 降到 0.0930。GPTQ 相对 RTN 下降了约 69.4%。

5.4 GPTQ：二阶误差补偿

GPTQ (Generalized Post-Training Quantization) 是面向大模型的二阶训练后权重量化方法。与 RTN 独立量化每个权重不同，GPTQ 在量化当前列后立即调整尚未量化的列，使后续列尽可能抵消当前列引入的输出误差。

5.4.1 Hessian 矩阵的物理意义

GPTQ 的核心是利用输入激活的统计特性来衡量各列之间的关联强度。给定校准数据 $X \in R^{N \times d}$ (N 个 token, d 维输入), Hessian 矩阵定义为：

$$H = \frac{2}{N} X^T X \quad (5.1)$$

直觉 Hessian 的第 (i, j) 元素 $H_{i,j} = \frac{2}{N} \sum_n X_{n,i} X_{n,j}$ 衡量输入的第 i 列和第 j 列的相关性。如果两列经常同时激活（相关性强）， $H_{i,j}$ 大；如果两列独立， $H_{i,j}$ 小。GPTQ 用这个信息决定：量化第 j 列引入的误差如何传播到其他列，以及如何调整其他列来抵消这个误差。

5.4.2 Cholesky 分解与误差传播

直接对 Hessian 求逆数值不稳定（大模型中 Hessian 可能接近奇异），GPTQ 加入阻尼项后做 Cholesky 分解：

$$\lambda = \alpha \cdot \text{mean}(\text{diag}(H)), \quad H + \lambda I \xrightarrow{\text{Cholesky}} U^T U \quad (5.2)$$

其中 U 是上三角矩阵。量化第 j 列时：

1. 先计算归一化误差： $e_j = \frac{w_j - q_j}{U_{j,j}}$ ，其中 w_j 是原始权重列， q_j 是量化后的整数列。
2. 再沿 U 的第 j 行更新剩余列： $w_k \leftarrow w_k - e_j \cdot U_{j,k}$ ，对所有 $k > j$ 。

这表示：量化第 j 列产生的误差 e_j ，通过 $U_{j,k}$ 的权重传播到后续的每一列，在权重层面做预补偿。等价地，最终输出的误差被最小化。

例 用一个小例子理解误差补偿。假设有 3 列权重 w_0, w_1, w_2 ：

1. **RTN**：独立量化三列， $q_0 = \text{round}\left(\frac{w_0}{s_0}\right)$, $q_1 = \text{round}\left(\frac{w_1}{s_1}\right)$, $q_2 = \text{round}\left(\frac{w_2}{s_2}\right)$ 。误差互不影响。
2. **GPTQ**：量化 w_0 得到 q_0 ，计算误差 $e_0 = \frac{w_0 - q_0}{U_{0,0}}$ 。然后调整 $w_1 \leftarrow w_1 - e_0 U_{0,1}$, $w_2 \leftarrow w_2 - e_0 U_{0,2}$ 。接着量化调整后的 w_1 得到 q_1 ，计算误差 $e_1 = \frac{w_1' - q_1}{U_{1,1}}$ ，再调整 $w_2 \leftarrow w_2' - e_1 U_{1,2}$ 。最后量化 w_2'' 得到 q_2 。

GPTQ 的每一列量化都考虑了之前列的误差，并通过 Hessian 信息做了预补偿。最终三列的总输出误差比 RTN 小得多。

5.4.3 dead columns 与校准数据

在实际大模型中，某些列在校准数据中从未被激活（输入在这些维度上恒为零），对应的 Hessian 对角元素为零，导致除零错误。GPTQ 的处理是：对完全未激活的列，只将对应 Hessian 对角元素设为 1（而非 0），让这些列的量化退化为 RTN，不影响其他列的补偿。

Lab 5 实测发现，未修正 dead columns 的 GPTQ (gptq-nodead) 的 Δ NLL 为 0.2027，不通过阈值。修正后降到 0.0930，下降约 54.1%。这说明 dead columns 处理对精度至关重要。

校准数据的选择也影响精度。Lab 5 收集 4096 个有效输入 token 构造 Hessian。校准数据应该代表性好、覆盖面广，否则 Hessian 无法准确反映真实输入分布。太少 token 会导致 Hessian 估计不准；太多 token 增加计算成本。4096 是精度与成本的折中。

5.5 W4A16 的显存分析

W4A16 的 4 bit 只描述量化 Linear 的 packed weight，不能覆盖整个模型。对称 per-group 量化下，每个 Linear 除了每权重 0.5 byte 的 qweight，还要为每组 128 个输入通道保存一个 FP16 scale。对形状为 $(d_{\text{out}}, d_{\text{in}})$ 的 Linear，主要存储量为：

$$M_{\text{linear}} = \frac{d_{\text{out}} \times d_{\text{in}}}{2} + 2d_{\text{out}} \left\lceil \frac{d_{\text{in}}}{128} \right\rceil \text{ bytes} \quad (5.3)$$

例 Gemma4-12B 有 328 个 Linear 模块使用 packed INT4, tied BF16 embedding 为 1.875 GiB。量化 Linear 理论上约占 5.24 GiB，加上 embedding 合计约 7.12 GiB，与实测 7.2 GiB checkpoint 一致。这就是 W4A16 的显存账本：权重压缩了 4 倍，但 scales 和未量化的 embedding/output weight 仍占空间。

6 量化推理的瓶颈分析

量化不是免费的午餐。把权重压缩到低精度后，推理时需要反量化恢复为浮点才能参与计算。这个反量化步骤本身可能成为瓶颈。

6.1 反量化瓶颈

在 Lab 5 的参考实现中，每次 forward 都要先把 packed INT4 权重展开为 uint8，再执行减法和乘法物化完整 BF16 权重，最后才做 GEMM。这意味着每次推理都重复一遍完整的反量化。

例 Lab 5 的 profiler 数据：反量化过程产生多个中间张量 (unpack、view、broadcast、sub、mul)，总 CUDA 时间约 2.2 s。相比之下，实际执行矩阵乘的 aten::mm 仅 0.222 s，占 8.79%。两者比例约 10:1。INT4 解包对应的 aten::__rshift__ 和 aten::bitwise_and 各调用 656 次。

这说明当前 decode 的首要算子瓶颈不是 attention，而是重复的权重解包与反量化。如果把反量化优化掉，理论上可以再提升 10 倍。

直觉 为什么会这样？因为 W4A16 的 GEMM 不是原生的 INT4 GEMM，而是先反量化到 FP16 再做 FP16 GEMM。每次 forward 都要为所有 328 个 Linear 做一遍反量化。而权重是不变的，反量化结果每次都一样，却被重复计算。这就是融合 dequant-GEMM 的动机：不要物化中间的 FP16 权重，而是在寄存器中反量化后立即与输入 tile 相乘。

6.2 融合 dequant-GEMM 原理

融合 dequant-GEMM 内核的核心思想是：不把整个权重矩阵反量化后存储，而是每次只从显存读取一个 packed INT4 tile (K 维度的一小块)，在寄存器中用位运算解包、乘以 scale 反量化，然后立即与输入 tile 相乘累加到 FP32 accumulator。这样反量化结果从不写入显存，只存在于寄存器中。

Lab 5 的 Triton 实现包含两个关键优化：

1. **Scale 加载合并**：取 `BLOCK_K = group_size = 128`，一个 K tile 内所有列属于同一 group，scale 是常数。优化前对每列都加载一次 scale，优化后每 K tile 仅加载一次。这把 scale 的加载量减少了 128 倍。
2. **无分支解包**：原始实现对每个元素用条件分支选择低或高 nibble (`tl.where(is_low, packed & 0x0F, packed >> 4)`)。优化后用移位量自动选择：`(packed >> ((k_offs & 1) * 4)) & 0xF`，用位运算替代条件分支，消除 warp divergence。

融合 dequant-GEMM 的效果取决于 M (batch 或 token 数)。在 decode 阶段 M 很小 (如 $M = 1$)，GEMM 是访存密集的，融合后避免了物化中间权重，收益大。在 prefill 阶段 M 很大 (如 $M = 2000$)，GEMM 是计算密集的，cuBLAS 针对大矩阵优化更好，融合 kernel 反而慢。这就是 HybridQuantizedLinear 的动机。

6.3 HybridQuantizedLinear: 按 M 切换策略

Lab 5 的最终方案是混合策略：当 $M \leq 64$ (decode 场景) 使用融合 Triton kernel，当 $M > 64$ (prefill 场景) 使用 dequant 加 cuBLAS fallback。

例 实测交叉点： $M = 1$ 时 fused 比 dequant 快 1.25 倍； $M = 2000$ 时 dequant 比 fused 快 25.6 倍。交叉点在 $M \approx 64$ 。这是因为 fused kernel 的优势在于避免物化中间权重 (访存优化)，当 M 小时 GEMM 本身访存多计算少，融合收益大；当 M 大时 GEMM 计算量大，cuBLAS 的 tiling 优化更重要，融合的访存优势被稀释。

最终端到端性能：融合 async 配置下 BS1 decode 15% 更快，但 prefill 29% 更慢。这验证了混合策略的必要性：没有单一 kernel 在所有 M 下都最优。

6.4 Lab 2 的 requantize 瓶颈对比

Lab 2 的 W8A8 也有类似的中间精度转换问题：SwiGLU 激活函数需要在浮点域执行，因此 INT8 GEMM 的 INT32 输出必须反量化为浮点，执行激活后再量化回 INT8。这个 requantize 是 Lab 2 的热点之一。

两种瓶颈的本质对比：

维度	Lab 2 requantize	Lab 5 dequant
触发原因	SwiGLU 非线性运算	INT4 到 FP16 的精度转换
频率	每个 FFN stage 一次	每次 forward 全部 Linear
优化方向	减少 requantize 次数	融合 dequant-GEMM
能否消除	不能（激活必须浮点）	可以（融合进 GEMM）

7 量化格式全景与前沿

7.1 四类量化格式

量化格式不止整数一种。根据数值网络的分布方式，可以分为四大类：

格式	位宽	网格分布	特点
Binary/Ternary	1-2 bit	符号乘以 scale, 或 $\{-1, 0, +1\}$	极端压缩，精度损失大
定点整数 INT-b	b bit	均匀网格, 2^b 个等间距值	最常见, 硬件支持广
浮点 FP4/FP8	4/8 bit	对数网格, 零附近密集	同位宽下精度更好
两级缩放	4 bit + scale	FP4 值加 FP8 细粒度 scale	近乎无损的 4 bit

直觉 三种网格分布的本质区别：INT 是均匀的，每隔固定距离放一个网格点；FP 是对数的，零附近密、远处疏。因为神经网络的权重和激活大多集中在零附近（正态分布），所以 FP 格式在同样位数下通常比 INT 更准确。Binary/Ternary 走到极端，只保留符号或三值，靠 scale 恢复幅度。

7.2 NVFP4：两级缩放的 4 bit 浮点

NVFP4 是 NVIDIA 推出的 4 bit 浮点格式，采用两级缩放：每 16 个 FP4 值配一个 FP8 尺度（E4M3，精细的尺子），每个张量配一个 FP32 全局尺度（全局修剪）。FP4 采用 E2M1 格式，有 15 个可表示值： $\{0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\}$ 。

例 两级缩放的直觉：全局尺度确定大范围，每 16 个值内的 FP8 尺度做精细调整。就像用一把粗尺子量整体长度，再用一把细尺子量每小段的精确值。实测在 Blackwell GPU 上，W4A4 GEMM 的峰值推理吞吐是 FP8 的 3 倍，内存流量是 BF16 的 1/4。DeepSeek-R1 从 FP8 量化到 NVFP4，精度损失低于 1%（MMLU-Pro 85 $\rightarrow$ 84）。这就是两级缩放的威力：4 bit 几乎无损。

7.3 异常值问题

当模型参数超过 67 亿时，激活的某些通道会出现异常值（outliers）：这些通道的激活值比其他通道大几十甚至上百倍。如果简单地把所有通道统一量化，异常值会撑大 scale，导致其他通道的精度严重下降。这是大模型量化的核心难题。

应对方案经历了从 2022 到 2025 年的演进：

方法	思路	年份
LLM.int8()	混合精度：异常值通道保留 FP16，其余 INT8	2022
SmoothQuant	把激活异常值平滑转移到权重上再量化	2023
QuaRot	随机正交矩阵旋转，分散异常值到所有维度	2024
SpinQuant	学习到的正交矩阵旋转，效果更好	2024
SageAttention2	逐线程 INT4 量化，专门针对注意力计算	2025

直觉 旋转量化的数学原理是旋转不变性：注意力分数 QK^T 在正交变换 R 下保持不变，因为 $(QR)(R^TK^T) = Q(RR^T)K^T = QK^T$ 。所以可以选一个合适的 R ，把权重和激活都旋转，使异常值被均匀分散，然后再量化。推理时只需在输入端乘 R 、输出端乘 R^T ，开销很小。

Lab 5 的 GPTQ 通过 *per-group* 量化间接缓解了异常值问题：每 128 列共享一组 *scale*，异常值只影响它所在的组，不会撑大全局 *scale*。但这是权重量化的解法，激活异常值仍需 SmoothQuant 或旋转量化等方法。三个实验都没有直接处理激活异常值，因为 Lab 2 的 INT8 精度足够（127 个值），Lab 4.5 不量化激活（模拟 FP64），Lab 5 的 W4A16 也不量化激活。

8 跨实验对比与总结

8.1 统一对比表

维度	Lab 2 (W8A8)	Lab 4.5 (逐级量化)	Lab 5 (W4A16 GPTQ)
目标	加速 MoE 推理	模拟 FP64 GEMM	压缩模型显存
量化对象	权重 + 激活	FP64 矩阵元素	仅权重
位宽	INT8	INT8（多级叠加）	INT4
粒度	权重 per-matrix, 激活 per-token	per-tensor	per-group (G=128)
scale 计算	$\max \frac{\text{abs}}{127}$	递推 $s_{i+1} = \frac{s_i}{254}$	$\max \frac{\text{abs}}{7}$
误差补偿	无（独立量化）	无（逐级残差）	GPTQ 二阶补偿
中间转换	requantize (SwiGLU 间)	多级 INT8 叠加	dequant (INT4 $\rightarrow$ FP16)
硬件	Intel AMX / VNNI	INT8 Tensor Core	FP16 Tensor Core
精度指标	RMSE lt 2e-3	L2 rel. err. 1.22e-7	Δ NLL lt 0.16
实测加速	9.5x	36.8x	满足 10 GiB 限制

8.2 量化方法选择逻辑

1. 瓶颈在显存 $\rightarrow$ W4A16 (只量化权重, 压缩比最大)。如 Lab 5 的 12B 模型装不进 10 GiB。
2. 瓶颈在算力 $\rightarrow$ W8A8 (权重和激活都量化, INT8 GEMM 直接加速)。如 Lab 2 的 MoE FFN。
3. 需要高精度计算但硬件不支持 $\rightarrow$ 逐级量化 (低精度叠加模拟高精度)。如 Lab 4.5 的 FP64 模拟。
4. 有校准数据且需要极致压缩 $\rightarrow$ GPTQ (二阶补偿控制 INT4 精度损失)。如 Lab 5 的量化。
5. 无校准数据且只需快速量化 $\rightarrow$ RTN (独立舍入, 最简单)。精度损失大, 适合 INT8 或更高位宽。
6. 激活有异常值 $\rightarrow$ SmoothQuant 或旋转量化。适用于 6.7B 以上模型的 W8A8 场景。

这三个实验有一个共同的教训: 量化不是简单地把 *float* 转成 *int*, 而是一个系统工程。选位宽要看瓶颈 (显存 *vs* 算力), 选粒度要看数据分布 (静态 *vs* 动态), 选算法要看精度要求 (RTN *vs* GPTQ), 选系统方案要看 M 大小 (*fused vs dequant+cuBLAS*)。每一步的选择都影响最终效果。

9 本章你将学会

1. 写出线性量化和反量化的公式 $q = \text{round}(\frac{x}{s} + z)$ 、 $\hat{r} = s(q - z)$, 解释 s 和 z 的物理含义。
2. 区分对称量化 ($z = 0$, 范围 $[-8, 7]$) 和非对称量化 ($z \neq 0$, 范围 $[0, 15]$), 说明各自适用场景。
3. 比较 per-tensor、per-channel、per-group、per-token、per-matrix 五种粒度的优缺点, 解释 Lab 2 为何对权重用 per-matrix 而对激活用 per-token。
4. 解释为什么 INT8 点积必须用 INT32 累加 (2^{14} 单项乘积, INT16 第二项溢出)。
5. 分析 W8A8 与 W4A16 的根本区别: 量化对象、GEMM 精度、requantize 需求、适用瓶颈。
6. 描述 Lab 2 中 requantize 的产生原因 (SwiGLU 非线性运算) 和它与 Lab 5 dequant 瓶颈的异同。
7. 推导逐级量化的数学: $x \approx \sum q_i s_i$, scale 递推 $s_{i+1} = \frac{s_i}{254}$, 解释 254 的由来 (2×127)。
8. 说明 quant_splits 优化为何能把量化时间降低 61% (跳过未使用的量化层)。
9. 解释 RTN 的不足 (独立量化, 忽略误差累积) 和 GPTQ 的改进 (二阶补偿, 列间误差传播)。
10. 推导 GPTQ 的 Hessian $H = \frac{2}{N} X^T X$ 的物理意义和 Cholesky 分解的作用。
11. 说明 dead columns 问题及其处理 (对角元素设 1, 退化为 RTN) 对精度的影响 (Δ NLL 从 0.2027 降到 0.0930)。
12. 分析反量化成为瓶颈的原因 (dequant vs GEMM 比例 10:1) 和融合 dequant-GEMM 的原理 (寄存器内反量化, 不物化中间权重)。
13. 解释 HybridQuantizedLinear 的交叉点 ($M \approx 64$) 和按 M 切换策略的必要性。

14. 比较四类量化格式（Binary/INT/FP/两级缩放）的网格分布差异，解释 NVFP4 两级缩放为何让 4 bit 近乎无损。

10 要点速查

概念	英文	一句话要点
线性量化	Linear Quantization	$q = \text{round}(\frac{r}{s} + z)$, $\hat{r} = s(q - z)$, 两个参数定天下
对称量化	Symmetric Quantization	$z = 0$, 范围 $[-8, 7]$, 计算简单, 不对称数据浪费范围
非对称量化	Asymmetric Quantization	$z \neq 0$, 范围 $[0, 15]$, 用满范围, 多存 z
per-tensor	Per-tensor	整张量一个 scale, 开销最小精度最差
per-group	Per-group	每 G 列一个 scale, Lab5 取 $G = 128$
per-token	Per-token	每个 token 一个 scale, Lab2 用于激活
per-matrix	Per-matrix	每个矩阵一个 scale, Lab2 用于权重
W8A8	Weight 8-bit Activation 8-bit	权重和激活都量化, INT8 GEMM 直接加速
W4A16	Weight 4-bit Activation 16-bit	仅权重量化, GEMM 时反量化到 FP16
INT32 累加	INT32 Accumulation	INT8 乘积 2^{14} , INT16 第二项溢出, 必须 INT32
requantize	Requantize	SwiGLU 之间 INT8 $\rightarrow$ FP32 $\rightarrow$ INT8 转换
逐级量化	Multi-split Quantization	$x \approx \sum q_i s_i$, scale 按 254 递减
scale 递推	Scale Recursion	$s_0 = \frac{X_{\max}}{127}$, $s_{i+1} = \frac{s_i}{254}$
quant_splits	quant_splits	只量化使用的层, 时间降 61%
INT4 打包	uint8_little_nibble	两个 INT4 打包进一个字节, 低 nibble 偶数高 nibble 奇数
RTN	Round to Nearest	独立舍入, 不考虑误差累积, INT4 下精度差
GPTQ	GPTQ	二阶补偿, 量化列后调整剩余列, Δ NLL 降 69%
Hessian	Hessian	$H = \frac{2}{N} X^T X$, 衡量列间相关性
Cholesky 分解	Cholesky Decomposition	$H + \lambda I = U^T U$, 数值稳定求逆
dead columns	Dead Columns	校准中未激活的列, 对角设 1 退化 RTN
NLL	Negative Log-Likelihood	语言模型预测指标, Δ NLL < 0.16
反量化瓶颈	Dequant Bottleneck	dequant 2.2s vs GEMM 0.22s, 比例 10:1

融合 dequant-GEMM	Fused Dequant-GEMM	寄存器内反量化, 不物化中间权重
HybridQuantizedLinear	Hybrid Linear	$M \leq 64$ 用 fused, $M > 64$ 用 dequant+cuBLAS
NVFP4	NVFP4	4 bit 浮点加两级缩放, 3 倍吞吐近乎无损
异常值	Outliers	6.7B 以上激活通道异常值, 撑大 scale 降精度
SmoothQuant	SmoothQuant	激活异常值转移到权重上
旋转量化	QuaRot/SpinQuant	正交旋转分散异常值, 旋转不变性

11 小结

我们从三个实验的三种量化路径出发，系统梳理了量化技术的数学基础、设计选择和工程瓶颈。

量化的数学核心是线性映射 $q = \text{round}(\frac{r}{s} + z)$ ，两个参数 s 和 z 决定了一切。对称量化令 $z = 0$ 简化计算，非对称量化用满范围但多存一个参数。粒度选择是精度与开销的权衡：per-tensor 最省但最不准，per-group（Lab 5 的 $G = 128$ ）是 INT4 的经验折中，per-token（Lab 2 的激活）适配动态分布。误差来源包括舍入误差、截断误差和分布失配，衡量指标因场景而异：NLL 衡量模型能力（Lab 5），MSE 和 L2 relative error 衡量数值精度（Lab 2/4.5）。

三种量化路径各有定位。Lab 2 的 W8A8 同时量化权重和激活，用 AMX 原生 INT8 GEMM 加速 MoE 推理，瓶颈在 SwiGLU 之间的 requantize。Lab 4.5 的逐级量化把 FP64 分解为多个 INT8 分量叠加，scale 按 254 递减，quant_splits 优化跳过未使用层，实现 36.8 倍加速。Lab 5 的 W4A16 只量化权重，GPTQ 用 Hessian 二阶补偿控制 INT4 精度损失，dead columns 处理和 Cholesky 分解保证数值稳定。

推理瓶颈的分析揭示了量化的隐藏代价：反量化可能比 GEMM 本身慢 10 倍。融合 dequant-GEMM 在寄存器内反量化避免物化中间权重，但只在小 M （decode）下有效；大 M （prefill）下 cuBLAS 更优。HybridQuantizedLinear 按 M 切换策略是最终的工程解法。

前沿的 NVFP4 用两级缩放让 4 bit 浮点近乎无损，异常值问题用旋转量化（QuaRot/SpinQuant）分散到所有维度。这些技术正在把量化的极限推向更低 bit、更高精度。

一句话总结：量化是精度、显存、算力三者之间的三角博弈，选择取决于瓶颈在哪。

讲义基于 HPC101 课程 Lab 2、Lab 4.5、Lab 5 的量化实验内容编写